One of the challenges of quickly stepping through all seven colors is that we need to alternate between one color on, and then two colors on, and then back to one, moving through a specific sequence. For example, if we start with red, then we need to add green to make yellow, and then turn off red to leave green. To keep track of these alternating colors, or pin numbers, we set up two local variables in our `loop()` function:

```
int newPin = 0;
int oldPin = 0;
```

We'll come back to these variables in a moment. The next thing we need to do is to read the state of the switch to see if it has been triggered or not. The challenge here is that because switches are mechanical devices, whenever a switch is triggered, the metal contacts inside the switch have a tendency to "bounce," creating false or multiple triggers. To get around this, we can very quickly read the state of the digital input twice with a short pause between each reading and compare the results. If the two readings are the same, chances are pretty good that the switch has indeed been triggered. If not, it's probably a false reading. This is called **debouncing**. To do this we use the following:

```
int bounce1 = digitalRead(switchPin);
delay(25);
int bounce2 = digitalRead(switchPin);
```

In this section of code, we declare a variable called `bounce1` and assign it the value of the switch reading, pause for a very short amount of time, 25 milliseconds, and then declare a second variable called `bounce2` and assign it the value of a second reading. That's just enough time to ensure a good reading from the switch. We can then compare these readings in a `while` loop:

```
while ((bounce1 == bounce2) && (bounce1 == LOW)) {
```

For as long as both readings are the same, and they are both `LOW` (the tilt switch is normally closed so it only opens, creating a `LOW` signal in our circuit when the switch is tilted) the following code in the `while` loop will continuously run. Once the switch has been triggered, we begin the process of quickly blinking through each of the seven colors. To do this we bring back the following two variables:

```
oldPin = newPin;
newPin++;
```

What these two lines do is to assign the old value of `newPin` to `oldPin` and then increment `newPin` by 1. This way `newPin` is always one value ahead of `oldPin`. From the declaration of these two variables earlier, each start at 0, `oldPin` stays at 0 while `newPin` becomes 1. These values will be used as the index in the array and correspond to pin numbers for the RGB LED, so index 0 is pin 9, index 1 is pin 10, and index 2 is pin 11. Each time through the loop these numbers will increment by 1. If `newPin` ever reaches 3, an index that does not exist, this line will reset `newPin` to the value 0, as follows:

```
if (newPin == 3) newPin = 0;
```

This might make more sense when we look at the following business end of this loop:

```
digitalWrite(rgb[oldPin], HIGH);
delay(time);
digitalWrite(rgb[newPin], HIGH);
delay(time);
digitalWrite(rgb[oldPin], LOW);
```